

工具调用

使用 Kimi API 的联网搜索功能

📄 复制页面



在之前的章节中（[使用 Kimi API 完成工具调用](#)），我们详细说明了如何通过 Kimi API 的工具调用

`tool_calls` 特性完成 Kimi 大模型的联网搜索功能，我们回顾一下之前实现过程的内容：

1. 使用 JSON Schema 格式定义工具，在联网搜索的场合，我们定义了 `search` 和 `crawl` 两个工具；
2. 通过 `tools` 参数将定义好的 `search` 和 `crawl` 提交给 Kimi 大模型；
3. Kimi 大模型会根据当前聊天的上下文，选择调用 `search` 和 `crawl`，并生成相关参数，以 JSON 格式输出；
4. 使用 Kimi 大模型输出的参数，执行 `search` 和 `crawl` 函数，并将函数执行结果提交给 Kimi 大模型；
5. Kimi 大模型根据工具执行结果，给予用户回复；

在实现联网搜索的过程中，我们需要自己实现 `search` 和 `crawl` 函数，这其中可能包括：

1. 调用搜索引擎接口，或自己实现内容搜索；
2. 获取搜索结果，包括 URL 和摘要等信息；
3. 根据 URL 获取网页内容，可能需要针对不同的网站应用不同的读取规则；
4. 将获取的网页内容清洗并整理成模型便于识别的格式，例如 Markdown；
5. 处理各种错误和异常情况，例如无搜索结果、网页内容获取失败等；

实现上述这些步骤通常被认为是繁琐和富有挑战性的，我们的用户多次提出想要一个简单方便、开箱即用的“联网搜索”功能；因此我们基于 Kimi 大模型原有的工具调用 `tool_calls` 用法，提供了一个 Kimi 内置的工具函数 `builtin_function.$web_search`，以实现联网搜索功能。

的差别，我们将通过例子详细讲解如何调用 Kimi 内置的 `$web_search` 函数实现联网搜索功能，

KIMI 开放平台标注需要额外注意的事项。



`$web_search` 声明

与普通的 `tool` 不同，`$web_search` 函数并不需要提供具体的参数说明，仅需要在 `tools` 声明中 `type` 和 `function.name` 即可成功注册 `$web_search` 函数：

```
tools = [  
  {  
    "type": "builtin_function", # <-- 我们使用 builtin_function 来表示 Kimi 内置  
    "function": {  
      "name": "$web_search",  
    },  
  },  
]
```

`$web_search` 以美元符号 `$` 作为前缀，这是我们约定的表示 Kimi 内置函数的一种表达方式（在普通的 `function` 定义中，不允许出现美元符号 `$`），后续如果有其他 Kimi 内置函数，也将以美元符号 `$` 作为前缀。

当使用 `$web_search` 函数时，必须禁用模型的思考能力。

在声明 `tools` 时，`$web_search` 可以与其他普通的 `function` 共存，进一步地，`builtin_function` 与普通 `function` 是可以共存的，你可以在 `tools` 中既添加 `builtin_function`，又添加普通 `function`，或是同时添加 `builtin_function` 和普通 `function`。

接下来，让我们改造原先的 `tool_calls` 代码，来讲解如何执行 `tool_calls`。

`$web_search` 执行

以下是经过改造后的 `tool_calls` 代码：

 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

KIMI

开放平台



>

KIMI 开放平台

```
import json
```

```
>
from openai import OpenAI
from openai.types.chat.chat_completion import Choice
```

```
client = OpenAI(
    base_url="https://api.moonshot.cn/v1",
    api_key=os.environ.get("MOONSHOT_API_KEY"),
)
```

search 工具的具体实现，这里我们只需要返回参数即可

```
def search_impl(arguments: Dict[str, Any]) -> Any:
```

```
    """
```

在使用 Moonshot AI 提供的 search 工具的场合，只需要原封不动返回 arguments 即可，不需要额外的处理逻辑。

但如果你想使用其他模型，并保留联网搜索的功能，那你只需要修改这里的实现（例如调用搜索和获取网页内容等），函数签名不变，依然是 work 的。

这最大程度保证了兼容性，允许你在不同的模型间切换，并且不需要对代码有破坏性的修改。

```
    """
```

```
    return arguments
```

```
def chat(messages) -> Choice:
```

```
    completion = client.chat.completions.create(
```

```
        model="kimi-k2.6",
```

```
        messages=messages,
```

```
        max_tokens=32768,
```

```
        extra_body={
```

```
            "thinking": {"type": "disabled"}
```

```
        }, # 通过 extra_body 参数，传递额外请求体，从而禁用思考能力
```

```
        tools=[
```

```
            {
```

```
                "type": "builtin_function", # <-- 使用 builtin_function 声明 $web_
```

```
                "function": {
```

```
                    "name": "$web_search",
```

```
    return completion.choices[0]
```

```
>
```

```
def main():
    messages = [
        {"role": "system", "content": "你是 Kimi。"},
    ]

    # 初始提问
    messages.append({
        "role": "user",
        "content": "请搜索 Moonshot AI Context Caching 技术，并告诉我它是什么。"
    })

    finish_reason = None
    while finish_reason is None or finish_reason == "tool_calls":
        choice = chat(messages)
        finish_reason = choice.finish_reason
        if finish_reason == "tool_calls": # <-- 判断当前返回内容是否包含 tool_calls
            messages.append(choice.message) # <-- 我们将 Kimi 大模型返回给我们的 assistant
            for tool_call in choice.message.tool_calls: # <-- tool_calls 可能是多个
                tool_call_name = tool_call.function.name
                tool_call_arguments = json.loads(tool_call.function.arguments) #
                if tool_call_name == "$web_search":
                    tool_result = search_impl(tool_call_arguments)
                else:
                    tool_result = f"Error: unable to find tool by name '{tool_call_name}'"

            # 使用函数执行结果构造一个 role=tool 的 message，以此来向模型展示工具调用的结果
            # 注意，我们需要在 message 中提供 tool_call_id 和 name 字段，以便 Kimi
            # 能正确匹配到对应的 tool_call。
            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "name": tool_call_name,
                "content": json.dumps(tool_result), # <-- 我们约定使用字符串格式
```

KIMI 开放平台 `print(choice.message.content) # <-- 在这里，我们才将模型生成的回复返回给用户` 🔍 ⋮

```
if __name__ == '__main__':  
    main()
```

回顾上述代码，我们惊讶地发现，在使用 `$web_search` 函数时，其基本流程与普通的 `function` 并无区别，开发者甚至可以不用修改原先执行工具调用 `tool_calls` 的代码。而其中不一样并且尤其显得特别的地方在于，我们在实现 `search_impl` 函数时，并没有过多的搜索、解析、获取网页内容的逻辑，我们只是简单地将 Kimi 大模型生成的参数 `tool_call.function.arguments` 原封不动地返回即可完成工具调用 `tool_calls`，这是为什么呢？

事实上，正如 `builtin_function` 的名称所指示的那样，`$web_search` 是 Kimi 大模型内置的函数，其由 Kimi 大模型定义，也由 Kimi 大模型执行。其流程为：

1. 当 Kimi 大模型生成了 `finish_reason=tool_calls` 的响应时，表明 Kimi 大模型已经意识到当前需要执行 `$web_search` 函数，并且也已经做好执行 `$web_search` 的一切准备工作；
2. Kimi 大模型会将执行函数所必须得参数以 `tool_call.function.arguments` 的形式返回给调用方，但这些参数并不由调用方执行，调用方只需要将 `tool_call.function.arguments` 原封不动地提交给 Kimi 大模型，即可由 Kimi 大模型执行对应的联网搜索流程；
3. 当用户将 `tool_call.function.arguments` 使用 `role=tool` 的 `message` 提交时，Kimi 大模型随即开始执行联网搜索流程，并根据搜索和阅读结果生成可供用户阅读的消息，即 `finish_reason=stop` 的 `message`；

关于兼容性的说明

Kimi API 提供的联网搜索功能，旨在不破坏原有 API 和 SDK 兼容性的前提下，提供一种可靠性高的大模型联网搜索解决方案，其完全兼容 Kimi 大模型原有的工具调用 `tool_calls` 特性，这意味

可在不破坏代码整体结构的情况下完成：

KIMI 开放平台



1. 将 `$web_search` 的 `tool` 定义修改成你自己实现的 `tool` 定义（包括 `name`、`description` 等），这可能需要在 `tool.function` 中添加额外的说明信息以告知模型具体需要生成哪些参数，你可以在 `parameters` 字段中添加任意你需要的参数信息；
2. 修改 `search_impl` 函数的实现，在使用 Kimi 提供的 `$web_search` 时，你只需要原封不动返回入参 `arguments` 即可，但如果你使用自己的联网搜索服务，你可能需要完整实现文章开头所提到的 `search` 和 `crawl` 功能；

完成上述步骤后，你就成功完成了从 Kimi 提供的联网搜索功能，迁移到自己实现的联网搜索功能的所有事项。

关于 Tokens 消耗

在使用 Kimi 提供的联网搜索函数 `$web_search` 时，搜索结果同样会被计入提示词所占用的 Tokens 中（即 `prompt_tokens`）。通常情况下，由于联网搜索的结果包含的内容众多，最终产生的 Tokens 消耗也会更多，为了避免在不知情的情况下消耗大量 Tokens，我们在生成 `$web_search` 函数的参数 `arguments` 时，会额外添加一个 `total_tokens` 字段，用于告知调用方，本次搜索内容总共占用的 Tokens 数量，这些 Tokens 将会在你完成整个联网搜索流程时计入 `prompt_tokens` 中，我们将使用具体的代码来展示如何获取这些 Tokens 消耗：

```
import json

>
from openai import OpenAI
from openai.types.chat.chat_completion import Choice
```

```
client = OpenAI(
    base_url="https://api.moonshot.cn/v1",
    api_key=os.environ.get("MOONSHOT_API_KEY"),
)
```

search 工具的具体实现，这里我们只需要返回参数即可

```
def search_impl(arguments: Dict[str, Any]) -> Any:
    """
```

在使用 Moonshot AI 提供的 search 工具の場合，只需要原封不动返回 arguments 即可，不需要额外的处理逻辑。

但如果你想使用其他模型，并保留联网搜索的功能，那你只需要修改这里的实现（例如调用搜索和获取网页内容等），函数签名不变，依然是 work 的。

这最大程度保证了兼容性，允许你在不同的模型间切换，并且不需要对代码有破坏性的修改。

```
    """
```

```
    return arguments
```

```
def chat(messages) -> Choice:
    completion = client.chat.completions.create(
        model="kimi-k2.6",
        messages=messages,
        max_tokens=32768,
        tools=[
            {
                "type": "builtin_function",
                "function": {
                    "name": "$web_search",
                },
            },
        ],
    )
```


KIMI 开放平台

```
usage = completion.usage
```

```
choice = completion.choices[0]
```

```
>
```

```
# =====
```

```
# 通过判断 finish_reason = stop, 我们将完成联网搜索流程后, 消耗的 Tokens 打印出来
```

```
if choice.finish_reason == "stop":
```

```
    print(f"chat_prompt_tokens:          {usage.prompt_tokens}")
```

```
    print(f"chat_completion_tokens:      {usage.completion_tokens}")
```

```
    print(f"chat_total_tokens:            {usage.total_tokens}")
```

```
# =====
```

```
return choice
```

```
def main():
```

```
    messages = [
```

```
        {"role": "system", "content": "你是 Kimi。"},
```

```
    ]
```

```
# 初始提问
```

```
messages.append({
```

```
    "role": "user",
```

```
    "content": "请搜索 Moonshot AI Context Caching 技术, 并告诉我它是什么。"
```

```
})
```

```
finish_reason = None
```

```
while finish_reason is None or finish_reason == "tool_calls":
```

```
    choice = chat(messages)
```

```
    finish_reason = choice.finish_reason
```

```
    if finish_reason == "tool_calls":
```

```
        # Handle reasoning_content field
```

```
        # Manually construct assistant message to ensure it includes reasoning
```

```
        message_dict = {
```

```
            "role": "assistant",
```

```
            "content": choice.message.content,
```

```
            "tool_calls": choice.message.tool_calls,
```

```
            "reasoning_content": getattr(choice.message, "reasoning_content",
```

KIMI

开放平台

```
for tool_call in choice.message.tool_calls:
    tool_call_name = tool_call.function.name
    tool_call_arguments = json.loads(
        tool_call.function.arguments)
    if tool_call_name == "$web_search":

        # =====
        # 我们将联网搜索过程中，由联网搜索结果产生的 Tokens 打印出来
        search_content_total_tokens = tool_call_arguments.get("usage",
        print(f"search_content_total_tokens: {search_content_total_tokens}")
        # =====

        tool_result = search_impl(tool_call_arguments)
    else:
        tool_result = f"Error: unable to find tool by name '{tool_call_name}'"

    messages.append({
        "role": "tool",
        "tool_call_id": tool_call.id,
        "name": tool_call_name,
        "content": json.dumps(tool_result),
    })

print(choice.message.content)

if __name__ == '__main__':
    main()
```

执行上述代码，获得如下返回结果：

```
chat_prompt_tokens:      13212 # <-- 代表包含了联网搜索结果的输入 Tokens 数量
completion_tokens:      295   # <-- 代表 Kimi 大模型根据联网搜索结果生成的 Tokens 数量
chat_total_tokens:       13507 # <-- 代表包含了联网搜索流程的请求消耗的总 Tokens 数量

# 此处省略 Kimi 大模型根据联网搜索结果生成的内容
```

关于模型大小的选择

另一个随之而来的问题是，当启用了联网搜索功能后，由于 Tokens 数量发生了较大的变化，超出了原本使用的模型上下文窗口，此时很可能触发一个 `Input token length too long` 报错信息。因此，在使用联网搜索功能时，我们建议选择模型 `kimi-k2.6`，以适应 Tokens 变化的情况，我们稍微改动 `chat` 函数的代码以使用 `kimi-k2.6` 模型：

```
def chat(messages) -> Choice:
    completion = client.chat.completions.create(
        model="kimi-k2.6",
        messages=messages,
        tools=[
            {
                "type": "builtin_function", # <-- 使用 builtin_function 声明 $web_
                "function": {
                    "name": "$web_search",
                },
            }
        ]
    )
    return completion.choices[0]
```

关于其他 tools

`$web_search` tools 可以与其他普通 tools 混合使用，你可以自由组合 `type=builtin_function` 和 `type=function` 的 tools。

 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

除了 Tokens 消耗外，我们还会对每次联网搜索收取一次调用费用，详情请见[计费](#)。

KIMI

开放平台



此页面对您有帮助吗？

 是

 否

[< Tool Calls 说明](#)

[官方工具集成 >](#)

